



GM-03 开发套件介绍

1.组成

GM-03开发套件并不是传统意义上的开发板！

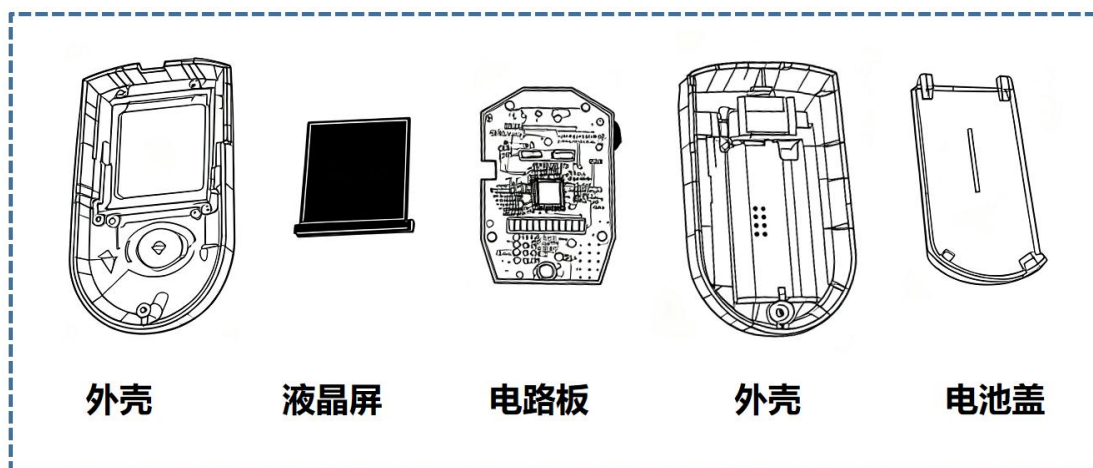
传统的开发板通常是将微控制器、基础外设（如LED、按键、串口）堆叠在一块简易电路板上，因此它们往往布局松散、缺乏工业设计。

相比之下，GM-03开发套件是一个革命性的概念：开发者拿到的不是一个实验性质的“半成品”，而是一个拥有紧凑尺寸、精致外观、成熟稳定性和完整功能的产品。

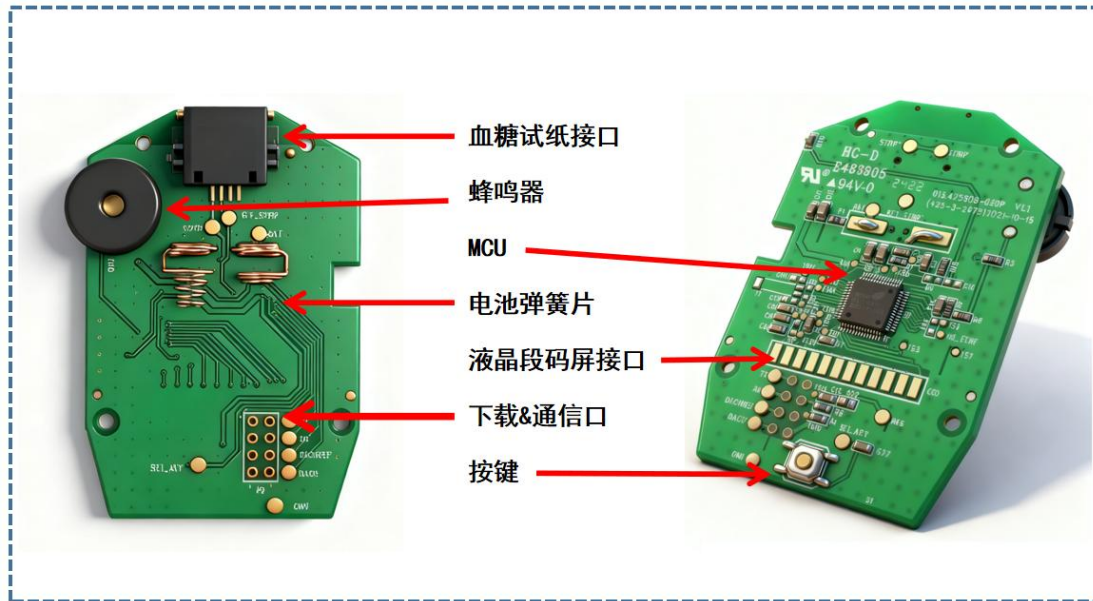


GM-03 开发套件由以下部件组成：

- ◆ **以 BH67F2472 为核心的电路板：**这个电路板绝非简单的“单片机+周边”。BH67F2472 是一款专为血糖仪等医疗设备设计的高集成度、高精度模拟前端 MCU。它集成了超低功耗设计、高精度 ADC（模数转换器）、生物传感器激励电路、液晶驱动等关键模块。
- ◆ **段码液晶屏：**这是为血糖仪量身定制的段码液晶屏，其显示内容如血糖数值、单位、时间、电池状态、错误提示符等，段码 LCD 以其极低的功耗著称。
- ◆ **塑料外壳：**外壳是区分“开发板”和“产品”的最直观标志。GM-03 的塑料外壳是专为血糖仪设计的，具有紧凑的人体工学设计，尺寸小巧，握持舒适，便于携带，让开发者体验真实的产品工业设计。



2.电路说明



GM-03 电路板非常独特，整个电路板上只有一个芯片！没错，只有一个芯片！

这个芯片完成了微小的生物电化学信号的采集和转换，完成了段码液晶屏的驱动，完成了温度测量、完成了按键和蜂鸣器的控制。这个神奇的芯片就是 **BH67F2472**！

BH67F2472 是 Holtek（合泰）推出的血糖仪专用 MCU，专为带 LCD 显示的血糖仪设计，以高精度、低功耗、多合一检测为核心优势，是开发紧凑型智能血糖仪的理想单芯片解决方案。BH67F2472 芯片片内资源如下：

CPU特性

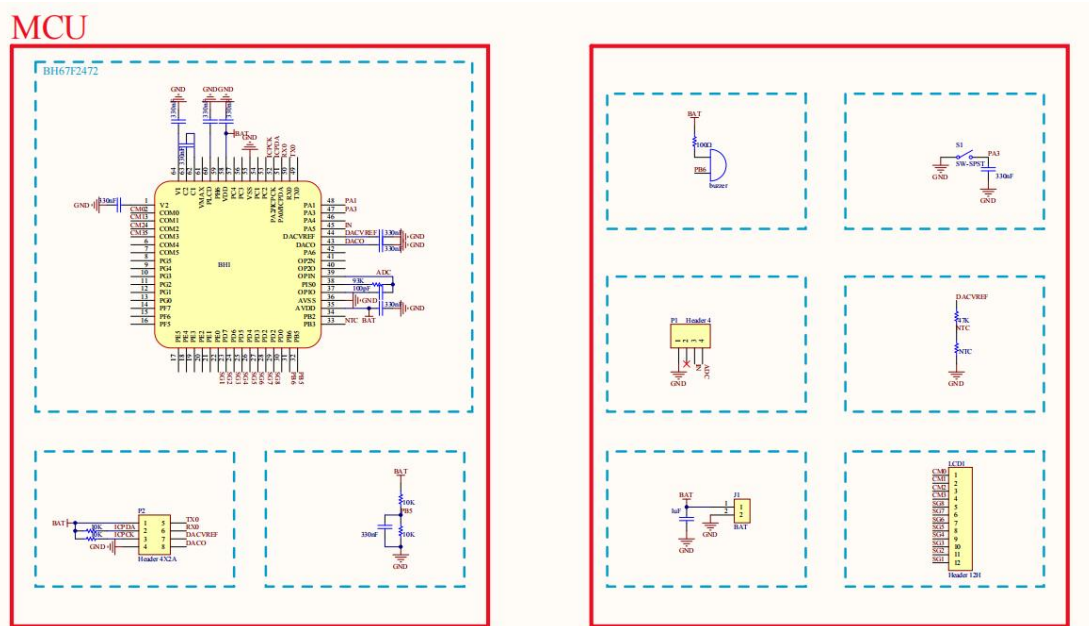
- 工作电压：
 - ◆ $f_{SYS}=4\text{MHz}$: 2.2V~5.5V
 - ◆ $f_{SYS}=8\text{MHz}$: 2.2V~5.5V
 - ◆ $f_{SYS}=12\text{MHz}$: 2.7V~5.5V
 - ◆ $f_{SYS}=16\text{MHz}$: 3.3V~5.5V
- $V_{DD}=5\text{V}$ ，系统时钟为 16MHz 时，指令周期为 0.25 μs
- 暂停和唤醒功能，以降低功耗
- 振荡器类型：
 - ◆ 内部高速 4/8/12MHz RC – HIRC
 - ◆ 内部低速 32kHz RC – LIRC
 - ◆ 外部高速晶振 – HXT
 - ◆ 外部低速 32.768kHz 晶振 – LXT
- 完全集成的内部振荡器无需外接元器件
- 多种工作模式：快速模式、低速模式、空闲模式和休眠
- 所有指令都可在 1~3 个指令周期内完成
- 查表指令
- 115 条功能强大的指令系统
- 16 层硬件堆栈
- 位操作指令

外设特性

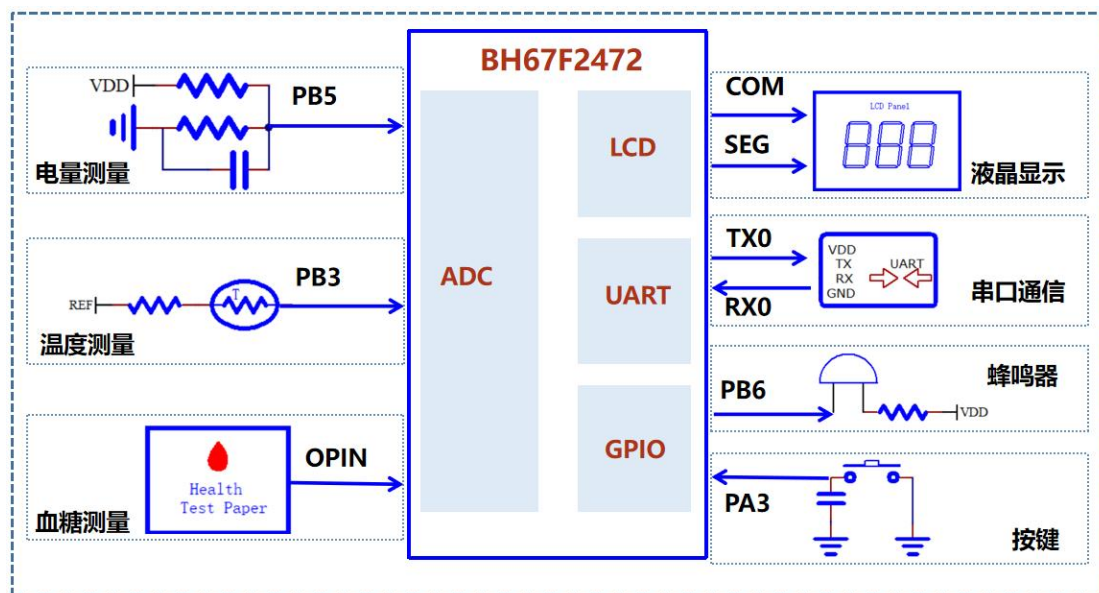
- Flash 程序存储器：32K×16
- 数据存储器：2048×8
- True EEPROM 存储器：2048×8
- 在线应用编程功能 – IAP
- 支持片上调试功能 – OCDS
- 看门狗定时器功能
- 58 个双向 I/O 口
- 4 个与 I/O 口共用引脚的外部中断输入
- 多个定时器模块用于时间测量、比较匹配输出、PWM 输出及单脉冲输出
- 双时基功能，用于产生固定时间的中断信号
- 血糖仪 AFE 集成电路
 - ◆ 6 个外部通道的 12-bit 分辨率 A/D 转换器
 - ◆ 内部参考电压发生器
 - ◆ 12-bit D/A 转换器
 - ◆ 2 个运算放大器
- LCD 驱动器功能
 - ◆ SEG × COM: 36×4, 34×6 或 32×8
 - ◆ 占空比类型: 1/4 Duty, 1/6 Duty 或 1/8 Duty
 - ◆ 偏压电平: 1/3 bias
 - ◆ 偏压类型: C 型
 - ◆ 波形类型: A 型或 B 型
- 两组通用串行接口模块 – USIM，用于 SPI, I²C 和 UART 通信
- 单个独立的串行外设接口 – SPI
- 低电压复位功能
- 封装类型：64/80-pin LQFP

GM-03 开发套件说明

GM-03 开发套件电路板的原理图如下:



GM-03 开发套件电路板的电路框图如下:

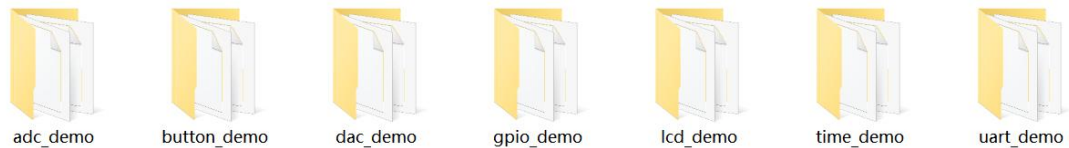


GM-03 开发套件包含 7 大功能:

- ◆ 液晶显示
- ◆ 串口通信
- ◆ 蜂鸣器
- ◆ 按键
- ◆ 电量测量
- ◆ 温度测量
- ◆ 血糖测量

3. DEMO 程序介绍

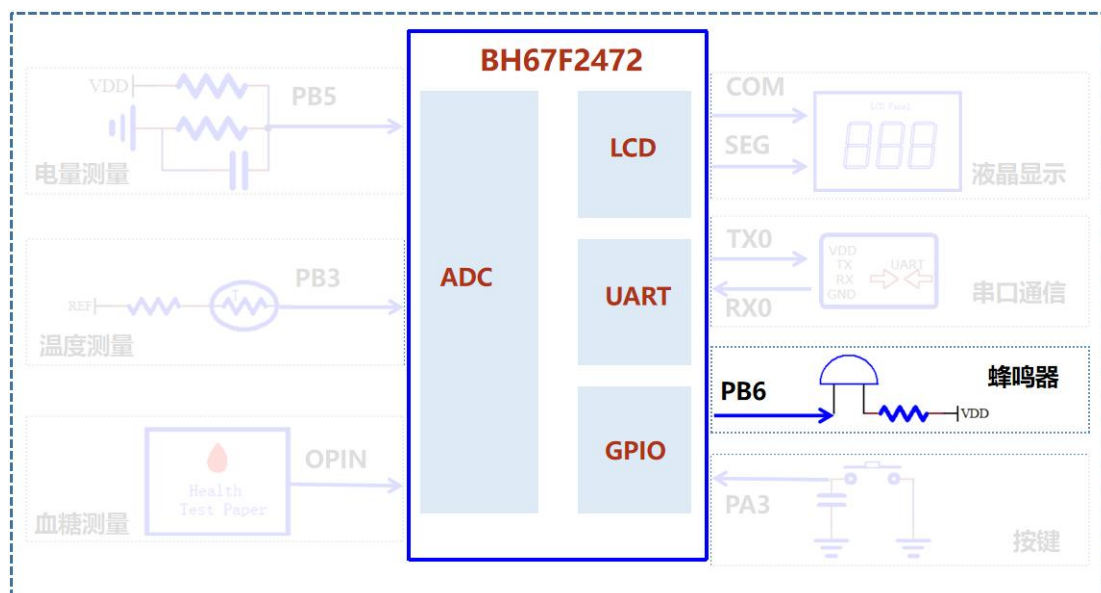
GM-03 开发套件提供了 7 个 demo 程序，每个 demo 用最简单的程序来实现相应功能，力求做到简单易懂。这 7 个 demo 程序分别是：ADC 程序、按键程序、DAC 程序、GPIO 程序、LCD 程序、定时器程序、串口通信程序。



3.1. gpio_demo 介绍

电路介绍

gpio_demo 程序通过控制 MCU 的 IO 口 PB6 的电平驱动蜂鸣器，让蜂鸣器发出声音，值得注意的是蜂鸣器不像 LED 那样，通过固定的高电平来点亮 LED，蜂鸣器需要一定频率（如 1KHZ）的高低交替电平信号控制，电路框图如下：



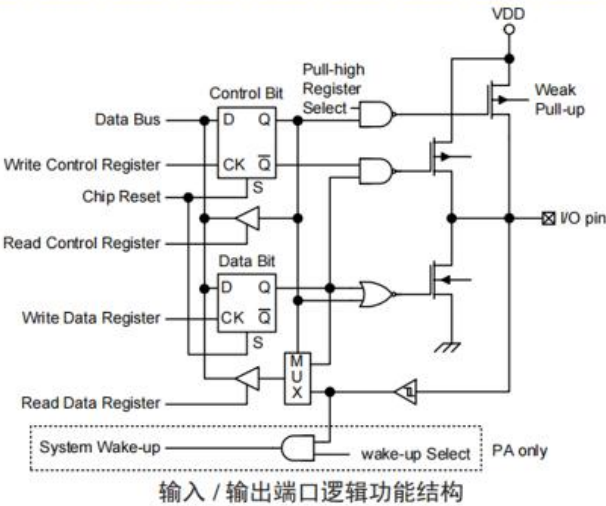
程序介绍

根据 GPIO 寄存器可知，通过配置 PBC 实现 GPIO 对应端口的输入输出选择，然后通过 PB6 寄存器输出高低电平信号。

• PxC 寄存器

Bit	7	6	5	4	3	2	1	0
Name	PxC7	PxC6	PxC5	PxC4	PxC3	PxC2	PxC1	PxC0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
POR	1	1	1	1	1	1	1	1

PxCn: I/O Px 口输入 / 输出类型选择位
0: 输出
1: 输入
PxCn 位用于控制引脚类型。这里的 x 可以是端口 A、B、C、D、E、F、G 或 H。但是，每个 I/O 端口实际有效位可能不同。



程序代码通过控制 PB6 的电平，实现蜂鸣器的驱动，让蜂鸣器发出声音。关键代码如下：

```

/*****
初始化io
*****/
void initialization(void)
{
    PIN_ModeOutput(pb,6);
}

/*****
设置IO输出
*****/
void gpio_set(unsigned char cmd )
{
    if(cmd == 0)
    {
        _pb6 = 1;    //输出高电平
    }
    else
    {
        _pb6 = 0;    //输出低电平
    }
}

```

```

/*****
主函数
*****/
void main(void)
{
    //配置系统时钟
    oscillators_cfg();
    //关闭看门狗
    _wdtc = 0b10101000;
    //设置为输出
    initialization();
    while(1)
    {
        gpio_set(1); //输出高电平
        delay_ms(10); //延时10ms
        gpio_set(0); //输出低电平
        delay_ms(10); //延时10ms
    }
}
/*****END*****/

```

技术细节

这里在定义宏的时候用了`##`，`##` 是一个预处理运算符，称为**连接运算符**（Token-Pasting Operator）。它用于预处理器处理阶段，可以将两个记号（tokens）连接起来，形成一个新的记号。

```

/*
*****
Define
*****
*/
#define PIN_ModeOutput(p, n)    { _##p##c##n = 0;}
#define PIN_ModeInput(p, n)    { _##p##c##n = 1;}
#define PIN_OutputHigh(p, n)   { _##p##c##n = 1;}
#define PIN_OutputLow(p, n)    { _##p##c##n = 0;}

```

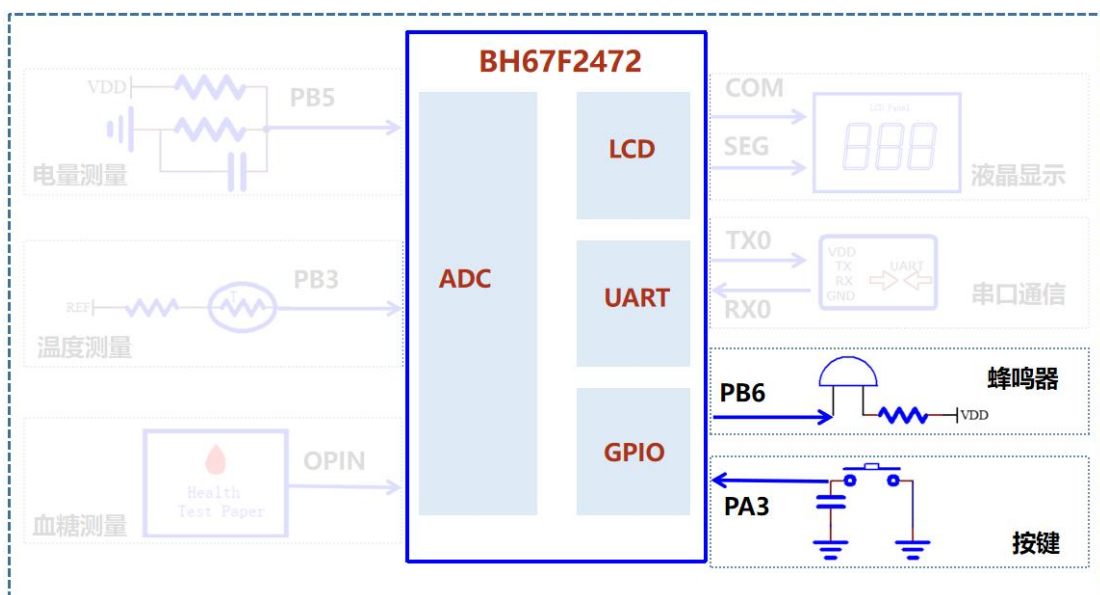
如：`#define PIN_ModeOutput(p, n) { _##p##c##n = 0;}`

`PIN_ModeOutput(pb,6)` 就等价于 `_pbc6 = 0`

3.2. button_demo 介绍

电路介绍

`button_demo` 程序通过读取 PA3 的电平信号来检测按键是否按下，如果检测到按键被按下程序会通过 PB6 驱动蜂鸣器发出声音，电路框图如下：



程序介绍

程序上电后 PA3 被默认配置成了输入模式，因此只需要将 PA3 设置成上拉模式即可，关键代码如下：

```

/*****
初始化io
*****/
void initialization(void)
{
    PIN_ModeOutput(pb,6);
    PIN_ModeInput(pa,3);
    PIN_PullHigh_Enable(pa,3);
}

/*****
设置IO输出
*****/
void gpio_set(unsigned char cmd )
{
    if(cmd == 0)
    {
        _pb6 = 1;      //输出高电平
    }
    else
    {
        _pb6 = 0;      //输出低电平
    }
}

/*****
按键扫描
*****/
unsigned char button_scanf(void)
{
    unsigned char ret = 0;
    if(KEY_DATA != 0)
    {
        delay_ms(10); //去抖动
        if (KEY_DATA != 0)
            ret = 1;
        else
            ret = 0;
    }
    return ret;
}

/*****
主函数
*****/
void main(void)
{
    //配置系统时钟
    oscillators_cfg();
    //关闭看门狗
    _wdtc = 0b10101000;
    //设置为输出
    initialization();
    while(1)
    {
        if(button_scanf() == 0)
        {
            gpio_set(1); //输出高电平
            delay_ms(10); //延时10ms
            gpio_set(0); //输出低电平
            delay_ms(10); //延时10ms
        }
    }
}

```

技术细节

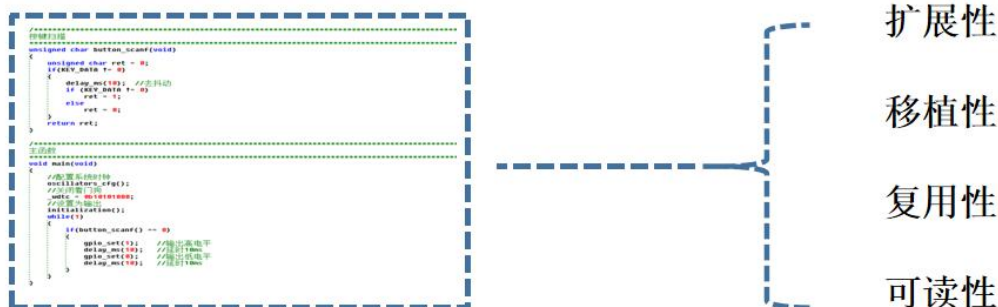
为了获得稳定的按键值，需要在程序中增加防抖动功能。

3.3. demo 介绍

待更新!!!

4. projtec 程序介绍

GM-03 开发套件除了提供简单的 **demo** 学习程序外，还将从编程规范、编程架构、编程技巧 3 个方面入手教大家如何写好一个“好程序”。一个优秀的程序必须要有良好的扩展性，移植性，复用性和可读性。



相信很多做嵌入式开发的朋友经常会遇到这种情况：项目需求会在开发中或者开发后变化，如果新的功能需求很多，导致软件全局很多地方需要修改，甚至有可能导致软件重写。造成这种结果的原因是，软件设计没有遵循软件设计原则，没有使用正确的设计模式和正确的软件架构。



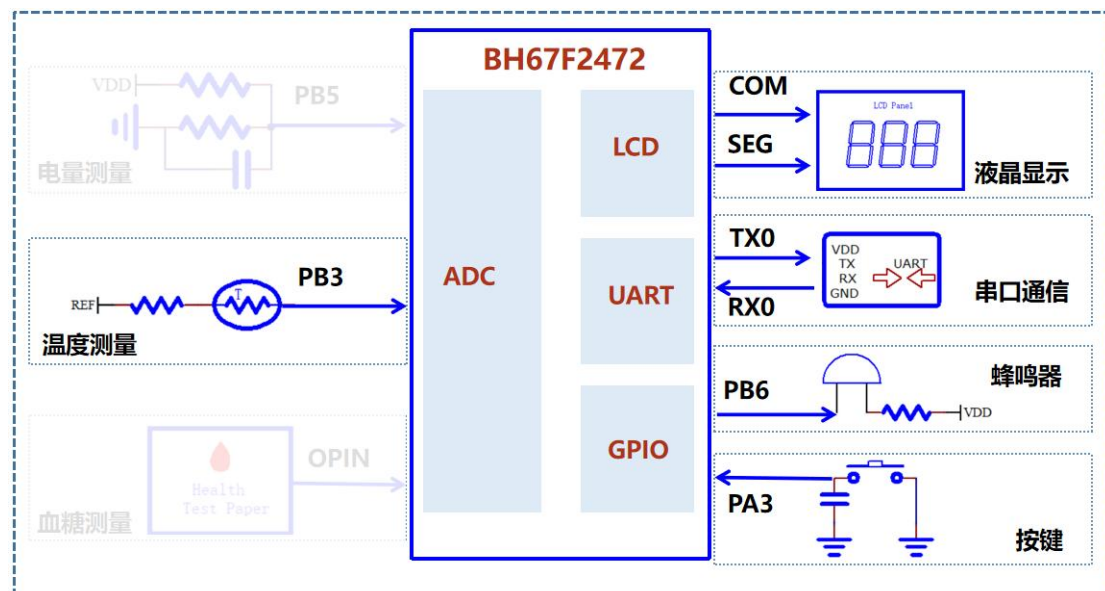
规则和方法繁多，新手往往难以融会贯通地使用到实际项目中，GM-03 开发套件提供了一个工程级别的程序 `ntc_lcd_project`，接下来基于 `ntc_lcd_project` 来介绍如何遵循软件设计原则，如何使用良好的设计模式和架构。

ntc_lcd_project 实现了“测量-处理-显示-交互”的完整流程。核心功能为温度测量，通过 NTC 热敏电阻将温度信号转化为电压信号，ADC 采集 NTC 节点电压，使用查表法计算得到温度。程序通过按键实现用户对显示内容的控制，短按按键实现循环切换显示模式：温度→血糖→电量→温度（血糖和电量显示值是一个虚拟值），每切换一次蜂鸣器会发出短鸣提示。

4.1. 电路介绍

程序使用了以下硬件资源：

- ◆ 按键：GPIO 口 PA3 连接按键，通过读取 PA3 的电平信号来检测按键是否按下；
- ◆ 蜂鸣器：GPIO 口 PB6 连接蜂鸣器，通过控制 PB6 的电平驱动蜂鸣器，让蜂鸣器发出声音；
- ◆ 液晶屏：LCD 驱动引脚 COM0~COM3，SEG1~SEG8 连接到了段码液晶屏，微控制器内部的 LCD 驱动控制器按照特定的扫描时序在 COM 和 SEG 线上产生驱动电压，点亮或熄灭液晶屏上特定的字段；
- ◆ 温度测量：ADC 通道 1 的 PB3 连接 NTC 热敏电阻，NTC 的电阻值随环境温度变化而变化，当温度变化时 NTC 上的分压值随之改变。ADC 通道读取 PB3 上的模拟电压值，实现温度测量；
- ◆ 串口通信：UART0 的 TX/RX 连接串口，实现输出调试打印信息。



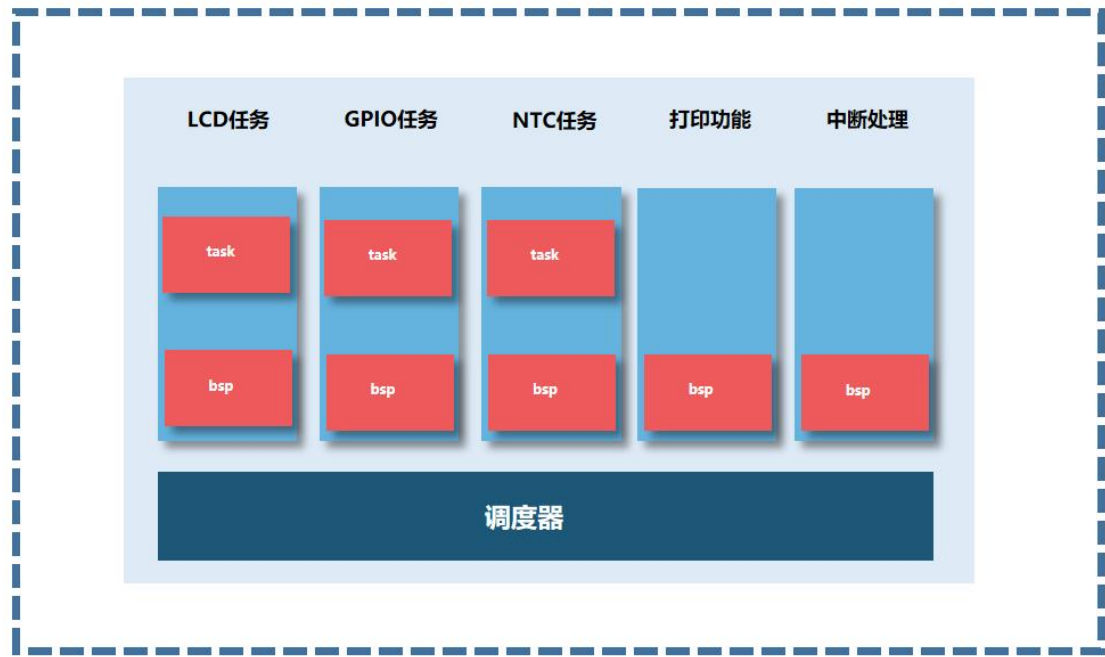
对硬件功能进行分类：

- ◆ GPIO 类，包含按键和蜂鸣器功能；
- ◆ LCD 类，包含液晶显示功能；
- ◆ ADC 类，包含模拟量测量功能。

4.2. 程序介绍

分模块设计

程序采用了模块化设计方法，每个功能独立成一个模块，每个单独的模块采用了分层设计。软件框架图如下：

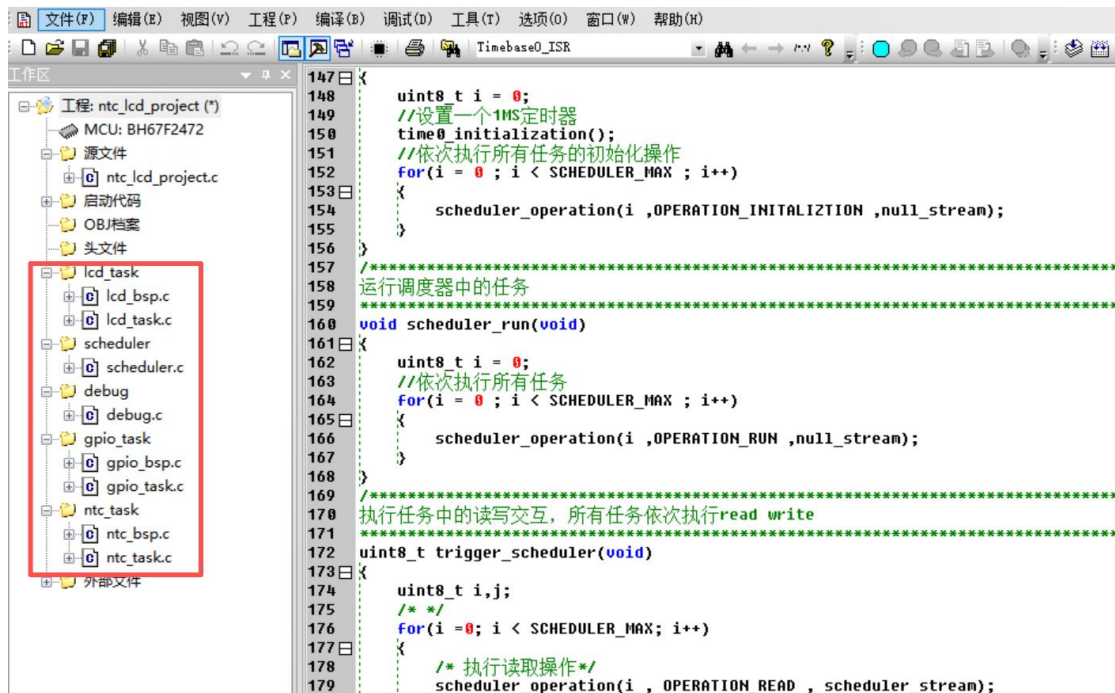


软件使用了模块化设计的方法，模块化设计核心思想就是“分而治之”，就是把一个复杂的问题分解为若干个简单的问题，然后逐个解决。ntc_lcd_project 程序包含以下三个任务：

- ◆ 任务一 GPIO 任务，GPIO 口 PA3 连接按键，GPIO 口 PB6 连接蜂鸣器，程序通过按键实现用户对显示内容的控制，短按按键实现循环切换显示模式：温度→血糖→电量→温度，每切换一次蜂鸣器会发出短鸣提示。
- ◆ 任务二 LCD 任务，程序控制微控制器内部的 LCD 驱动控制器点亮或熄灭液晶屏上特定的字段，实现 3 位 7 段数字的显示，同时段码液晶屏还可以显示不同数据的单位。
- ◆ 任务三 NTC 任务，ADC 通道 1 的 PB3 连接 NTC 热敏电阻，NTC 的电阻值随环境温度变化而变化，当温度变化时 NTC 上的分压值随之改变。ADC 通道读取 PB3 上的模拟电压值，实现温度测量。

GM-03 开发套件说明

模块化设计提高了软件系统的扩展性，模块可以根据需求布署和删除，模块化设计遵循了单一原则，软件工程源码中模块的划分如下：



调度器

RTOS 通常需要额外的内存开销用于任务栈、内核数据结构以及提供任务调度。由于 BH67F2472 有限的计算资源（如 RAM、ROM 容量较小）和相对较低的运算性能，无法有效地承载一个完整的实时操作系统（RTOS）运行环境。为了在资源受限的条件下实现多任务逻辑的轮转执行，开作者设计并实现了一个精简的轮询式任务调度器。

HOLTEK 开发环境所使用的 C 编译器不支持函数指针，函数指针是构建动态任务调用机制的常用且高效手段，缺失函数指针实现调度器将变得比较笨拙，只能使用枚举量和 switch 语句实现，在 scheduler 文件中实现了一个轮询执行的“伪调度器”，关键代码如下：

```
/*
*****
Typedef
*****
*/
//确保SCHEDULER_MAX在校举中最后一个位置
typedef enum {
    SCHEDULER_LCD,
    SCHEDULER_GPIO,
    SCHEDULER_NTC,
    SCHEDULER_MAX,
} scheduler_def;
```

```

void scheduler_operation(uint8_t scheduler ,uint8_t cmd ,uint8_t* stream)
{
    switch(scheduler)
    {
        //执行LCD任务
        case SCHEDULER_LCD:
            lcd_task_operation(cmd ,stream);
            break;
        //执行GPIO任务
        case SCHEDULER_GPIO:
            gpio_task_operation(cmd ,stream);
            break;
        //执行温度测量任务
        case SCHEDULER_NTC:
            ntc_task_operation(cmd ,stream);
            break;
        default:
            break;
    }
};

/*****
将调度器中的任务进行初始化
*****/
void scheduler_initialization(void)
{
    uint8_t i = 0;
    //设置一个1MS定时器
    time0_initialization();
    //依次执行所有任务的初始化操作
    for(i = 0 ; i < SCHEDULER_MAX ; i++)
    {
        scheduler_operation(i ,OPERATION_INITIALIZATION ,null_stream);
    }
}

```

这种设计实现的调度器被称为“伪调度器”，因为这个调度器有以下特点：

- ◆ 任务执行是顺序执行、非抢占执行。一个任务必须主动执行完毕并返回（break 出 case）后，调度器才能切换到下一个任务，不存在由中断或系统调用触发的任务强制切换。
- ◆ 静态绑定：任务与枚举值、case 分支是静态编译时绑定的，缺乏运行时动态创建、删除或修改任务列表的能力。
- ◆ 轻量级：其实现极其简洁，仅需一个枚举变量、一个 switch 语句和若干函数调用，几乎不消耗额外的 RAM 资源（栈空间除外），代码体积（ROM）也很小，完美契合资源受限环境。

分层设计

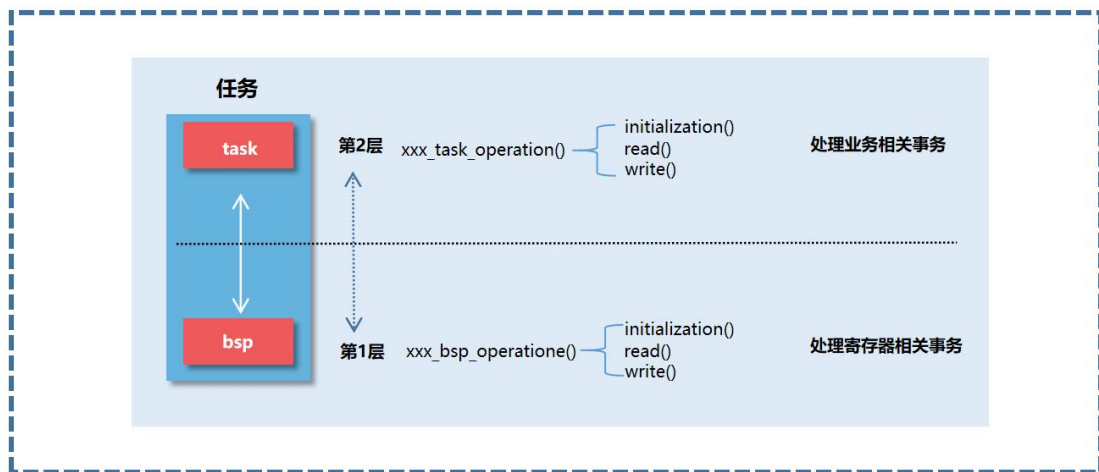
每一个任务都采用了分层设计，分层设计的核心思想也是“分而治之”，分层设计将软件功能水平分割成合理的多个子系统，软件中紧密关联的部分被集中放在一个层内。

分层架构有以下优点：

- ◆ 每一层都把一个具体功能抽象化。
- ◆ 可以降低代码的相互依赖程度，更改代码时影响的层很少。
- ◆ 层可以被复用。

程序中的每个功能模块采用了 2 层的分层设计，第 1 层处理 MCU 寄存器相关操作，第 2 层处理驱动控制和逻辑控制，分层设计提高了软件系统的移植性，如果项目更换了 MCU 那么只用修改第 1 层，如果更改了业务逻辑那么只用修改第 2 层。

分层架构框图如下：



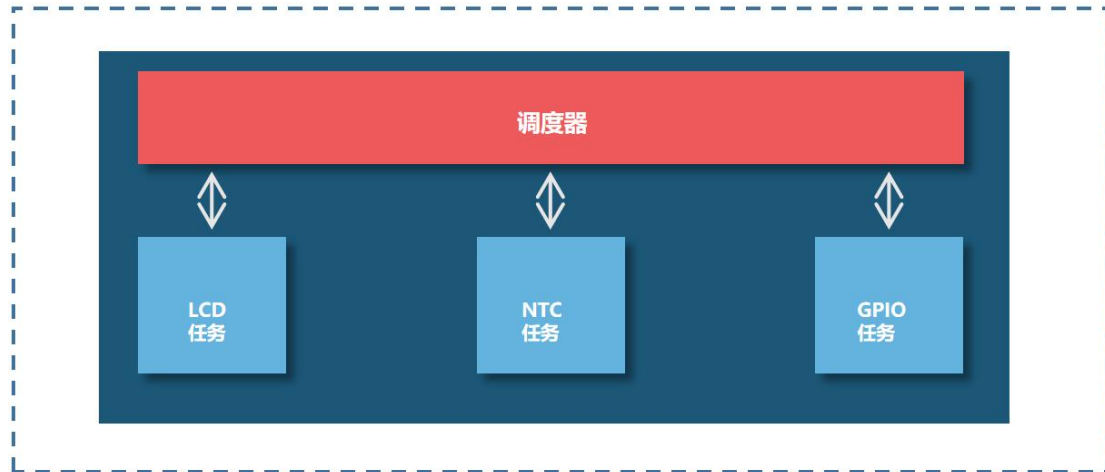
以 GPIO 任务为例，GPIO 的 BSP 层的接口函数是 `gpio_bsp_operation`，上层文件可以通过 `gpio_bsp_operation` 函数完成 GPIO 的寄存器初始化、读操作、写操作。

软件工程源码中 GPIO 任务的 BSP 层代码如下：

```
101 后台连续运行
102 *****
103 static void run(uint8_t*stream)
104 {
105 }
106
107 /*****
108  BSP级操作对象,由于编译器不支持函数指针,使用状态机调用函数
109 *****/
110 void gpio_bsp_operation(uint8_t cmd ,uint8_t*stream)
111 {
112     switch(cmd)
113     {
114         case OPERATION_READ:
115             read(stream);
116             break;
117         case OPERATION_WRITE:
118             write(stream);
119             break;
120         case OPERATION_INITIALIZATION:
121             initialization(stream);
122             break;
123         case OPERATION_RUN:
124             run(stream);
125             break;
126         default:
127             break;
128     }
129 }
130 *****
131 END
```

隔离设计

程序中的任务相互隔离,所有任务只与调度器进行数据交互,调度器将消息推送给相应任务。各个任务之间的信息交互模式如下:



这种设计模式为中间者模式。在中间者模式,对象之间不能直接通信,而是间接地通过中间者进行通信。中间者收到信息后,再将信息转发给相关对象,这样减少了对对象之间的相互依赖。中间者模式有以下优点:

- ◆ 对象之间是松耦合。
- ◆ 将多对多的关系通过中间者转换成一对一的关系。
- ◆ 修改一个对象,不需要考虑其它对象通信适应问题。

这种设计减少了任务之间的耦合,提高了软件的扩展性,消息交互代码如下:

```

/*****
执行任务中的读写交互,所有任务依次执行read write
*****/
uint8_t trigger_scheduler(void)
{
    uint8_t i,j;
    /* */
    for(i = 0; i < SCHEDULER_MAX; i++)
    {
        /* 执行读取操作*/
        scheduler_operation(i , OPERATION_READ , scheduler_stream);
        /*依次写入其他任务 */
        for(j = 0; j < SCHEDULER_MAX; j++)
        {
            /* 排除自身*/
            if(i != j)
                scheduler_operation(j ,OPERATION_WRITE ,scheduler_stream);
        }
        /*清空数据 */
        memset(scheduler_stream,0,sizeof(scheduler_stream));
    }
    return 0;
}
  
```

程序流程图

程序主要分为四个过程：

- ◆ 初始化系统时钟，配置 MCU 系统时钟为 8MHZ
- ◆ 执行调度器初始化动作，调度器依次调用所有任务中的 initialization 函数，执行各个任务初始化。
- ◆ 执行调度器依次调用所有任务 read 函数获取改任务输出信息，并将读取到的信息通过调用其他任务 write 函数写入执行操作。
- ◆ 执行调度器依次调用所有任务 run 函数，然后每个任务在后台运行。

程序流程图如下：

